
LECTURE NOTES

Lecture 15: Discrete Optimization

University

Simon Holm
April - 2026



DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1	The ROAR-NET API: Constructive Search	3
1.1	Community Detection Problem	3
2	Local Search Algorithms	4
2.1	Search space	4
2.2	Evaluation function	5
2.3	Initialization function	5
2.4	Neighborhood function	5
2.5	Transition model	5
2.6	Termination predicate	5
3	Metaheuristics	5
3.1	Evaluation function	6
3.2	Constraint-based local search	6
3.2.1	alldiff	6
3.3	Fundamental principles	7
3.4	Reservoir sampling	7

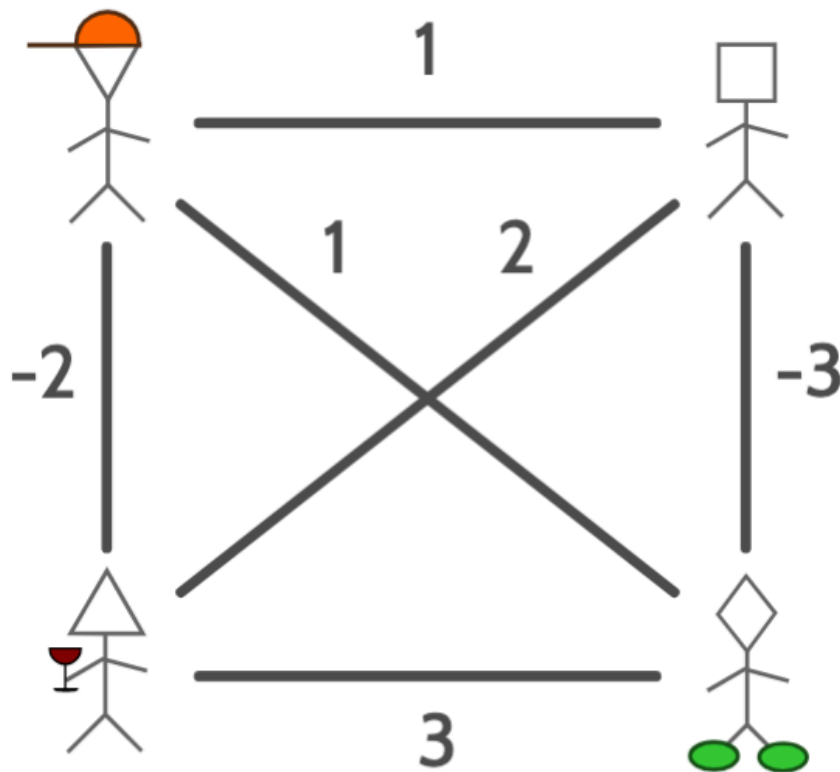
1 The ROAR-NET API: Constructive Search

1.1 Community Detection Problem

A fully-connected undirected graph

- vertices v must represent users
- Weighted edges represent the intensity of some attribute of their interaction

Positive weight indicate affinity between users, while negative edge weights indicate lack of affinity. For this groups of users connected mostly by positively weighted edges suggest the existence of a community involving those individuals.



Goal Partition the vertices into subsets while maximizing the total weight of the edges within the groups (cliques)

- **Clique-partitioning problem**

We need to define the problem and the API functions to be implemented

Problem

```
empty_solution(Problem) : Solution  
construction_neighbourhood(Problem) : Neighbourhood
```

Solution

```
objective_value(Solution) : double[0..1]  
lower_bound(Solution) : double[0..1]
```

Neighbourhood

```
moves(Neighbourhood, Solution) : Move[0..*]
```

Move

```
apply_move(Move, Solution) : Solution  
lower_bound_increment(Move, Solution) : double[0..1]
```

Given a (combinatorial) optimization problem Π and one of its instances π

2 Local Search Algorithms

Local search in a nutshell

- 1) Visit neighbors of a current solution
- 2) Decide whether to reject them or to accept one as the next solution
- 3) Repeat

At each solution one can then generate possible moves

- Enumeration (enumerate through the environment so and find all possible moves)
- Random sampling
 - With replacement
 - Without replacement

2.1 Search space

$$S(\pi)$$

- The search space is the set of all candidate solutions that may be visited during the solving process
- Only feasible solutions are considered in local search. The objective function is defined on the whole search space
- Where solution set $S'(\pi) \subseteq S(\pi)$
- Initial feasible solution(s) may be:
 - Generated at random
 - Obtained by constructive search

- Obtained heuristically in other ways

2.2 Evaluation function

$$f_{\pi} : S(\pi) \rightarrow \mathbb{R}$$

- Handles **soft constraints** and the objective function some thing like

$$\min f(x) - h(x)$$

2.3 Initialization function

$$\text{init} : \emptyset \rightarrow S(\pi)$$

- Can be seen as a probability distribution $P(S(\pi) \times M(\pi))$ over initial search positions and memory states

2.4 Neighborhood function

$$N_{\pi} S \rightarrow 2^{S(\pi)}$$

For each solution $s \in S(\pi)$ define a set of solutions that are in some sense close to s

2.5 Transition model

$$\text{step} : S(\pi) \times M(\pi) \rightarrow S(\pi) \times M(\pi)$$

- Takes a step
- Can be seen as a probability distribution $P(S(\pi) \times M(\pi))$ over subsequent, neighboring search positions and memory states

2.6 Termination predicate

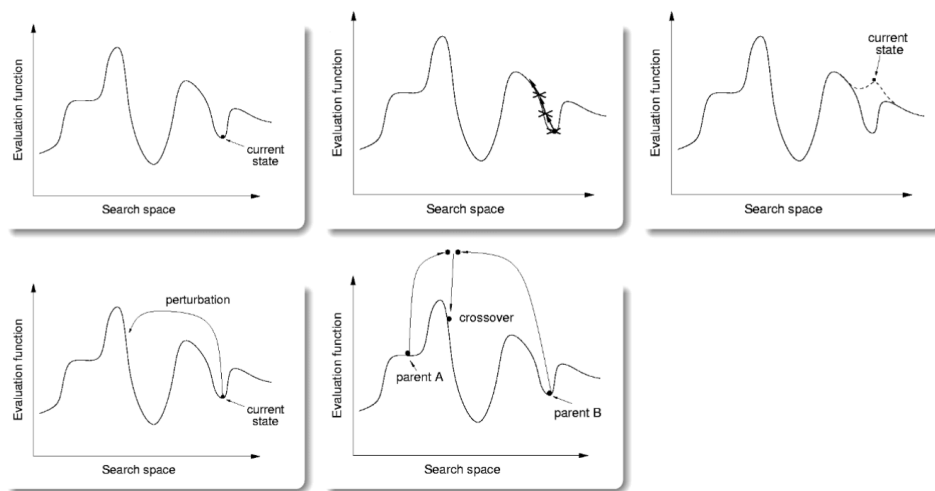
$$\text{terminate} : S(\pi) \times M(\pi) \rightarrow \{\top, \perp\}$$

- Determines the termination state for each search position and memory state

3 Metaheuristics

How to avoid getting trapped in bad local optima?

- Restart
- more complex neighborhood functions:
 - Variable Neighborhood Search and Large Scale Neighborhood Search
 - diversified neighborhoods + incremental algorithmics
- allow non-improving moves
 - Tabu Search: Online learning of moves, Discard undoing moves, Discard inefficient moves, Improve efficient moves selection
 - Simulated annealing



3.1 Evaluation function

Evaluation (or cost) function:

- function $f_{\pi} : S_{\pi} \rightarrow \mathbb{Q}$ that maps candidate solutions of a given problem instance π onto rational numbers (most often integer), such that global optima correspond to solutions of π ;
- used for assessing or ranking neighbors of current search position to provide guidance to search process.

Evaluation vs objective functions:

- **Evaluation function**: part of LS algorithm.
- **Objective function**: integral part of optimization problem.
- Some LS methods use evaluation functions different from given objective function (e.g., guided local search).

3.2 Constraint-based local search

If infeasible solutions are allowed, we count violations of constraints.

- decomposition-based violations
 - count number of violations, eg: `alldiff`
- variable-based violations
 - min number of variables that must be changed to satisfy c
- value-based violations
 - for constraints on number of occurrences of values
- arithmetic violations
- combinations of these

3.2.1 alldiff

`alldiff(x1, ..., xn)` Let a be an assignment with $V \setminus \{a(x-1), \dots, a(x_n)\}$ and $c_v = \#_{a(v,x)}$ be the number of violations in a Then possible definitions for violations are

- $\text{viol} = \sum_{v \in V} \mathbb{I}(\max\{c_v - 1, 0\} > 0)$ value-based; $\mathbb{I}$ indicator function.
- $\text{viol} = \max_{v \in V} \max\{c_v - 1, 0\}$ value-based
- $\text{viol} = \sum_{v \in V} \max\{c_v - 1, 0\}$ value-based
- # variables with same value, variable-based, here leads to same definitions as previous three

Arithmetic constraints:

- $l \leq r \rightsquigarrow \text{viol} = \max\{l - r, 0\}$
- $l = r \rightsquigarrow \text{viol} = |l - r|$
- $l \neq r \rightsquigarrow \text{viol} = 1$ if $l = r$, 0 otherwise

3.3 Fundamental principles

Simple properties:

- **search space size** $|S|$
- **reachability**: solution j is reachable from solution i if neighborhood graph has a path from i to j .
 - **strongly connected neighborhood graph**
 - **weakly optimally connected neighborhood graph**
- distance between solutions
- neighborhood size (ie, degree of vertices in neigh. graph)
- cost of fully examining the neighborhood
- relation between different neighborhood functions
(if $N_1(s) \subseteq N_2(s)$ for all $s \in S$ then N_2 dominates N_1)

3.4 Reservoir sampling

Store the best neighbor/neighbors. Iterate through neighborhood and replace with better neighbors. If another neighbor is equally good, replace at probability.